

Roteiro Explicativo dos Slides - Sistema Pata Feliz

Sistema Pata Feliz - Banco de Dados 2

Este roteiro acompanha os slides em **docs/relatorio-overleaf/slides.tex**. Ele foi escrito para ajudar na fala da apresentação e também para indicar onde cada parte aparece no código. A ideia não é decorar, mas entender o sistema o suficiente para explicar o que foi feito.

Slide 1 - Capa

O que falar

Apresente o nome do sistema: **Pata Feliz**. Explique que é um Sistema de Informação web para gestão de uma clínica veterinária.

Fale que o trabalho foi desenvolvido para Banco de Dados 2 e que o foco foi aplicar os conceitos do roteiro: CRUD, transações, consultas com junções, procedure armazenada, funções, triggers e múltiplas tabelas relacionadas.

Onde está no código

- ? Slides: **docs/relatorio-overleaf/slides.tex**
- ? Relatório: **docs/relatorio-overleaf/main.tex**
- ? Backend: **backend/server.js**
- ? Banco: **backend/database/schema-v2-postgres.sql**
- ? Frontend: **frontend/index.html** e **frontend/js/app.js**

Como explicar

Você pode dizer que o sistema foi dividido em três partes: interface no navegador, API no backend e banco PostgreSQL. Essa divisão ajuda a mostrar que o projeto é um sistema completo, e não apenas um script de banco.

Slide 2 - Objetivo do Sistema

O que falar

Explique que o objetivo é organizar a rotina de uma clínica veterinária. O sistema centraliza tutores, animais, veterinários, procedimentos, atendimentos, prontuários, bloqueios de agenda e lembretes.

Fale que o sistema resolve problemas práticos: evitar choque de horários, registrar histórico clínico, permitir agendamentos e gerar lembretes automáticos.

Onde está no código

- ? Objetivos no relatório: **docs/relatorio-overleaf/main.tex**, seção "Objetivos do Sistema".
- ? Rotas principais da API: **backend/routes/v2.js**.
- ? Tabelas principais: **backend/database/schema-v2-postgres.sql**.

Como explicar

O sistema manipula dados persistentes. Isso significa que as informações não ficam só na tela ou na memória: elas são gravadas no PostgreSQL e podem ser consultadas depois.

Slide 3 - Perfis de Usuário

O que falar

Explique os três perfis:

? **Administrador**: gerencia a clínica.

? **Veterinário**: acompanha sua agenda e registra prontuários.

? **Tutor/cliente**: cadastra animais, agenda atendimentos e visualiza informações permitidas.

Onde está no código

? Enum de tipo de usuário: **backend/database/schema-v2-postgres.sql**, tipo **tipo_usuario**.

? Login e token: **backend/routes/v2.js**, rota **POST /auth/login**.

? Controle de permissão: função **allow(...)** em **backend/routes/v2.js**.

? Menu do frontend por perfil: **frontend/js/app.js**.

Como explicar o código

No backend, depois do login, o sistema gera um token JWT com dados do usuário. Esse token guarda informações como **tipo_usuario**, **id_cliente** e **id_veterinario**.

A função **allow(...)** é usada nas rotas para bloquear acesso indevido. Por exemplo, algumas rotas só aceitam **admin**, outras aceitam **admin** e **veterinario**.

Slide 4 - Requisitos Funcionais

O que falar

Apresente as funcionalidades principais:

? Login por e-mail ou CPF.

? Cadastro e edição de clientes, animais, veterinários e procedimentos.

? Agendamento de atendimentos.

? Registro de prontuários.

? Lembretes automáticos.

? Relatórios.

Onde está no código

? Requisitos no relatório: **docs/relatorio-overleaf/main.tex**, seção "Requisitos Funcionais".

? Rotas de clientes: **backend/routes/v2.js**, **/clientes**.

? Rotas de animais: **backend/routes/v2.js**, **/animais**.

? Rotas de atendimentos: **backend/routes/v2.js**, **/atendimentos**.

? Rotas de prontuários: **backend/routes/v2.js**, **/prontuarios**.

? Rotas de relatórios: **backend/routes/v2.js**, **/relatorios**.

Como explicar

Cada requisito funcional virou uma ou mais rotas no backend. O frontend chama essas rotas usando **fetch**, e o backend consulta ou altera o banco.

Um exemplo simples: quando o usuário cadastra um animal na tela, o frontend envia os dados para **POST /animais**; o backend executa um **INSERT INTO animais**; e o PostgreSQL grava o registro.

Slide 5 - Arquitetura

O que falar

Explique a arquitetura em três camadas:

1. Frontend: telas e interação do usuário.
2. Backend: API, autenticação, permissões e regras.
3. PostgreSQL: dados, constraints, views, functions, procedure e triggers.

Onde está no código

- ? Frontend: **frontend/index.html**, **frontend/css/style.css**, **frontend/js/app.js**.
- ? Backend principal: **backend/server.js**.
- ? Rotas principais: **backend/routes/v2.js**.
- ? Conexão com banco: **backend/db.js**.
- ? Banco: **backend/database/schema-v2-postgres.sql**.

Como explicar o fluxo

Quando o usuário clica em algo no navegador, o frontend chama a API com **fetch**. O backend recebe a requisição, confere o token de login, verifica permissão, executa SQL no banco e devolve JSON para a tela.

Esse fluxo mostra integração real entre frontend, backend e banco.

Slide 6 - Modelo de Dados

O que falar

Explique que o banco tem várias tabelas relacionadas, atendendo ao requisito de manipular mais de uma estrutura de armazenamento.

As tabelas centrais são:

- ? **usuarios**
- ? **clientes**
- ? **veterinarios**
- ? **animais**
- ? **procedimentos**
- ? **atendimentos**
- ? **prontuarios**
- ? **lembretes**
- ? **bloqueios_agenda**
- ? **vacinas_aplicadas**

Onde está no código

- ? Criação das tabelas: **backend/database/schema-v2-postgres.sql**.
- ? Diagrama no relatório: **docs/relatorio-overleaf/main.tex**, seção "Diagrama Visual do Banco".

Como explicar o relacionamento

A tabela **usuarios** guarda os dados de acesso. Um usuário pode ser cliente ou veterinário. A tabela **animais** pertence a **clientes**. A tabela **atendimentos** liga quatro coisas importantes: animal, cliente, veterinário e procedimento.

O prontuário depende de um atendimento. Isso faz sentido porque o prontuário é registrado depois de uma consulta ou atendimento clínico.

Slide 7 - CRUD no Sistema

O que falar

O roteiro pede os comandos básicos SQL: **SELECT**, **INSERT**, **UPDATE** e **DELETE**. O sistema usa todos.

Onde está no código

- ? **SELECT**: listagens e relatórios em **backend/routes/v2.js**.
- ? **INSERT**: cadastro de cliente, animal, procedimento, atendimento e prontuário.
- ? **UPDATE**: edição de perfil, clientes, animais, procedimentos, status de atendimento e lembretes.
- ? **DELETE**: exclusão de bloqueio de agenda e lembrete.

Exemplos importantes:

- ? **POST /clientes**: insere em **usuarios** e **clientes**.
- ? **POST /animais**: insere em **animais**.
- ? **PATCH /atendimentos/:id/status**: atualiza status de atendimento.
- ? **DELETE /lembretes/:id**: remove lembrete.

Como explicar tecnicamente

CRUD significa:

- ? Create: criar dados com **INSERT**.
- ? Read: ler dados com **SELECT**.
- ? Update: alterar dados com **UPDATE**.
- ? Delete: excluir dados com **DELETE**.

No sistema, essas operações aparecem como rotas HTTP. Por exemplo, uma rota **POST** normalmente cria algo, uma rota **GET** consulta, uma rota **PUT** ou **PATCH** atualiza e uma rota **DELETE** remove.

Slide 8 - Conexão com o Banco

O que falar

Explique que a conexão com o banco fica centralizada em **backend/db.js**.

O projeto usa PostgreSQL com a biblioteca **pg**. Os dados de conexão vêm do arquivo **.env**, como host, usuário, senha, nome do banco e porta.

Onde está no código

- ? Arquivo de conexão: **backend/db.js**.
- ? Função PostgreSQL: **postgresPool()** em **backend/db.js**.
- ? Variáveis de ambiente: **backend/.env**.
- ? Uso das consultas: **db.query(...)** em **backend/routes/v2.js**.

Como explicar o código

No **db.js**, o sistema cria um **Pool**. Um pool é um conjunto de conexões reutilizáveis com o banco. Isso evita abrir uma conexão nova do zero a cada consulta.

Também existe uma função que converte ? em \$1, \$2, \$3, porque o código usa uma sintaxe parecida com MySQL, mas o PostgreSQL espera parâmetros numerados.

Exemplo de explicação:

> O backend não coloca o valor diretamente dentro do SQL. Ele manda o SQL com placeholders e os valores separados. Isso deixa a consulta mais organizada e segura.

Slide 9 - Transações

O que falar

Explique que transação é usada quando uma funcionalidade precisa executar mais de um comando SQL e tudo precisa dar certo junto.

O melhor exemplo é cadastro de cliente: primeiro insere em **usuarios**, depois insere em **clientes**. Se a segunda parte falhar, a primeira precisa ser desfeita.

Onde está no código

- ? Métodos de transação: **backend/db.js**, **beginTransaction**, **commit**, **rollback**.
- ? Cadastro de cliente: **backend/routes/v2.js**, rota **POST /clientes**.
- ? Edição de cliente: **backend/routes/v2.js**, rota **PUT /clientes/:id**.
- ? Edição de veterinário: **backend/routes/v2.js**, rota **PUT /veterinarios/:id**.
- ? Edição de perfil: **backend/routes/v2.js**, rota **PUT /perfil**.

Como explicar o código

O fluxo é:

1. **beginTransaction()**: inicia a transação.
2. Executa os comandos SQL.
3. **commit()**: confirma tudo se não houve erro.
4. **rollback()**: desfaz tudo se algo falhar.

Essa parte atende diretamente ao requisito do roteiro sobre transações.

Slide 10 - Consultas com Junções

O que falar

Explique que JOIN serve para juntar informações de várias tabelas em uma consulta só.

Use a agenda como exemplo: para mostrar um atendimento completo, é necessário juntar **atendimentos**, **animais**, **clientes**, **usuarios**, **veterinarios** e **procedimentos**.

Onde está no código

- ? View da agenda: **backend/database/schema-v2-postgres.sql**, **vw_agenda_atendimentos**.
- ? Função com JOINS: **listar_atendimentos(...)** em **backend/database/schema-v2-postgres.sql**.
- ? Histórico do animal: **buscar_historico_animal(...)**.
- ? Chamadas no backend: **backend/routes/v2.js**, rota **GET /atendimentos** e **GET /relatorios/historico-animal/:id**.

Como explicar tecnicamente

Sem JOIN, o backend teria que fazer várias consultas separadas e juntar os dados manualmente. Com JOIN, o PostgreSQL já retorna tudo consolidado.

Exemplo:

> A tabela **atendimentos** guarda os ids. O JOIN transforma esses ids em informações legíveis, como nome do animal, nome do tutor, nome do veterinário e nome do procedimento.

Slide 11 - Procedure e Funções Armazenadas

O que falar

Este slide é essencial para o roteiro. Explique que agora o sistema possui uma procedure real no PostgreSQL:

atualizar_status_lembrete(id, status, usuario)

Ela recebe parâmetros e atualiza o status de um lembrete.

Além da procedure, o sistema também usa funções armazenadas para buscas e relatórios.

Onde está no código

? Procedure real: **backend/database/schema-v2-postgres.sql**, **CREATE OR REPLACE PROCEDURE atualizar_status_lembrete**.

? Procedure no arquivo separado: **backend/database/functions-v2-postgres.sql**.

? Chamada da procedure: **backend/routes/v2.js**, rota **PATCH /lembres/:id/status**.

? Função de histórico: **buscar_historico_animal(p_id_animal BIGINT)**.

? Funções de listagem: **listar_clientes**, **listar_animais**, **listar_atendimentos**, **listar_prontuarios**, **listar_lembres**.

? Funções de relatório: **relatorio_servicos** e **relatorio_veterinarios**.

Como explicar a procedure

A procedure faz este papel:

1. Recebe o id do lembrete.
2. Recebe o novo status.
3. Recebe o id do usuário responsável.
4. Executa um **UPDATE** na tabela **lembres**.

O backend chama assim:

CALL atualizar_status_lembrete(?:bigint, ?::status_lembrete, ?::bigint)

Isso atende literalmente ao pedido de stored procedure com parâmetro.

Como explicar as funções

As funções são usadas quando o banco precisa retornar dados, por exemplo:

SELECT * FROM buscar_historico_animal(?)

Essa função recebe o id do animal e retorna o histórico clínico com JOINS. Ela é diferente da procedure porque retorna uma tabela para o backend.

Slide 12 - Triggers e Automações

O que falar

Explique que triggers são ações automáticas executadas pelo banco quando algo acontece em uma tabela.

No sistema, existem dois grupos de triggers:

1. Triggers para atualizar o campo **atualizado_em**.
2. Triggers para gerar lembres automáticos.

Onde está no código

? Função **set_atualizado_em()**: **backend/database/schema-v2-postgres.sql**.

? Triggers **trg_usuarios_atualizado_em**, **trg_clientes_atualizado_em**, **trg_animais_atualizado_em** e outras.

? Função **gerar_lembres_automaticos()**.

? Função **disparar_geracao_lembres()**.

? Triggers **trg_atendimentos_gerar_lembres** e **trg_vacinas_gerar_lembres**.

Como explicar tecnicamente

Quando uma linha é atualizada, a trigger chama uma função antes ou depois da alteração.

Exemplo:

> Quando um atendimento é inserido ou atualizado, a trigger dispara a função que gera lembres. Assim, o sistema pode criar alertas automaticamente sem depender de o usuário fazer isso manualmente.

Isso mostra programação dentro do banco de dados.

Slide 13 - Regras de Agendamento

O que falar

Explique as regras:

- ? Não pode agendar no passado.
- ? Não pode agendar com mais de um ano de antecedência.
- ? Segunda a sexta: 08:00 às 18:00.
- ? Sábado: 08:00 às 12:00.
- ? Domingo indisponível.
- ? Bloqueios de agenda impedem novos atendimentos.
- ? O mesmo veterinário não pode ter dois atendimentos sobrepostos.

Onde está no código

- ? Validação de data e horário: **validateSchedule(...)** em **backend/routes/v2.js**.
- ? Verificação de bloqueios: **hasScheduleBlock(...)** em **backend/routes/v2.js**.
- ? Criação de atendimento: rota **POST /atendimentos**.
- ? Restrição contra choque: **EXCLUDE USING GIST** em **backend/database/schema-v2-postgres.sql**, tabela **atendimentos**.

Como explicar tecnicamente

Parte da regra está no backend e parte está no banco.

O backend valida horário de funcionamento e datas inválidas antes de tentar inserir. Já o banco reforça a regra mais crítica: impedir dois atendimentos ativos no mesmo período para o mesmo veterinário.

Essa combinação é boa porque evita erro na aplicação e também protege o banco se alguém tentar inserir dados por outro caminho.

Slide 14 - Demonstração

O que falar

Este slide serve como roteiro da demonstração ao vivo.

Siga esta ordem:

1. Fazer login.
2. Mostrar dashboard.
3. Mostrar listagem de animais, clientes ou procedimentos.
4. Mostrar agenda.
5. Criar ou explicar um atendimento.
6. Mostrar prontuários.
7. Mostrar lembretes.
8. Mostrar relatórios.

Onde está no código

? Login: **POST** /api/auth/login.
? Dashboard: **GET** /api/dashboard.
? Agenda: **GET** /api/atendimentos.
? Prontuários: **GET** /api/prontuarios.
? Lembretes: **GET** /api/lembretes.
? Relatórios: **GET** /api/relatorios/servicos, **GET** /api/relatorios/veterinarios, **GET** /api/relatorios/historico-animais/:id.

Como explicar durante a demo

Quando mostrar uma tela, conecte a tela a um requisito do roteiro.

Exemplo:

> Esta tela de agenda usa SELECT com JOIN, porque ela mostra dados vindos de várias tabelas.

Outro exemplo:

> Ao marcar um lembrete como resolvido, o backend chama uma procedure real no PostgreSQL.

Slide 15 - Como Executar

O que falar

Explique os passos básicos para rodar:

1. Criar o banco **pata_feliz_v2**.
2. Executar **schema-v2-postgres.sql**.
3. Executar **seed-v2-postgres.sql**.
4. Executar **functions-v2-postgres.sql**, se precisar atualizar rotinas.
5. Rodar o backend.
6. Abrir o frontend.

Onde está no código

? Configuração do backend: **backend/.env**.
? Scripts SQL: **backend/database/**.
? Start do backend: **backend/package.json**, script **npm start**.
? Servidor Express: **backend/server.js**.
? Frontend: **frontend/index.html**.

Como explicar

O backend precisa saber como acessar o banco, por isso o **.env** guarda as credenciais. Depois que o banco está criado e populado com seed, o backend sobe na porta 3000 e o frontend acessa a API.

Slide 16 - Conclusão

O que falar

Retome os requisitos do roteiro e diga que o sistema atende:

- ? Sistema de Informação web.
- ? Operações de inserção, seleção, atualização e remoção.
- ? Mais de uma tabela relacionada.
- ? Conexão explícita com banco.
- ? Transações.
- ? Consulta com JOIN.

- ? Stored procedure real com parâmetro.
- ? Funções armazenadas.
- ? Triggers.

Onde está no código

- ? CRUD e rotas: **backend/routes/v2.js**.
- ? Transações: **backend/db.js** e rotas de clientes/perfil/veterinários.
- ? Procedure, funções e triggers: **backend/database/schema-v2-postgres.sql**.
- ? Frontend: **frontend/js/app.js**.

Como concluir bem

Finalize dizendo que o projeto não usa o banco apenas como depósito de dados. O PostgreSQL também participa da regra de negócio por meio de constraints, procedure, functions, views e triggers.

Slide 17 - Encerramento

O que falar

Agradeça e abra para perguntas.

Se perguntarem sobre stored procedure, responda:

> A procedure real é **atualizar_status_lembrete**. Ela recebe id do lembrete, status e usuário, e é chamada no backend com **CALL**.

Se perguntarem sobre função armazenada, responda:

> A função **buscar_historico_animal** recebe o id do animal e retorna o histórico clínico com JOINS.

Se perguntarem sobre trigger, responda:

> As triggers atualizam campos automaticamente e disparam geração de lembretes quando atendimentos ou vacinas mudam.

Se perguntarem sobre transação, responda:

> O cadastro de cliente usa transação porque insere em duas tabelas: **usuarios e clientes**.